DETERMLLM: Closing the Dominant Source of Batch Non-Determinism in BF16 LLM Inference

Anonymous Authors*

Abstract

BF16 LLM inference is non-deterministic: the same input at different batch sizes produces different output tokens. Yuan et al. [Yuan et al., 2025] (NeurIPS 2025) recently characterized the phenomenon on reasoning models and proposed LayerCast, a BF16-weights-FP32-compute baseline, at the cost of upcasting weights to FP32 on every matmul.

We target *token-wise* determinism (identical output tokens across batch sizes), and trace the dominant obstacle to cuBLAS's dynamic split-K selection in GEMM. We close that source with two backends that avoid LayerCast's bandwidth penalty. A precision amplification argument shows that FP32 reduction drives the non-associativity error below BF16's quantization step whenever $K < K_{\max} \approx 7{,}700$; we implement this as a cuBLASLt wrapper. A fixed-plan Triton kernel gives bit-exact batch-invariant GEMM output by construction at every K (i.e., the matmul result is bit-identical across batch sizes, which is a strictly stronger guarantee than token-wise). A hybrid backend routes small-N shapes (e.g. GQA K/V projections) to cuBLASLt and large-N shapes (FFN, lm_head) to Triton, recovering the best path at every shape.

On MATH500 with DeepSeek-R1-Distill-Qwen-7B, the hybrid backend matches the best Std@Acc across batch sizes in the comparison (tied with BF16 baseline and Triton at 0.0115, below LayerCast's 0.0141) while running $\mathbf{2.8}\times$ faster than LayerCast. Against vLLM 0.19's official `VLLM_BATCH_INVARIANT=1` mode (the public reference implementation of `batch_invariant_ops`), our SRP backend in HF eager achieves strict token-level bit-exactness across batch sizes (Avg_Std$=0$) at 279 s, versus vLLM-BI's probabilistic Avg_Std$= 2.55\times10^{-5}$ at 402 s—**stricter on determinism and 31% lower wall-clock on the same hardware**. Per-call GEMM overhead on DeepSeek shapes sums to -0.1% versus the non-deterministic BF16 baseline, compared with $+407\%$ for LayerCast. Legacy Llama-3.2-1B analysis corroborates at the GEMM level: the Triton backend cuts token mismatches $\mathbf{54}\times$ and log-probability variance $\mathbf{57}\times$, and the $K_{\max}$ bound predicts the 1B/8B/Phi-4 residual ordering. An M-padding fix closes the GEMV/GEMM boundary, adding $2.1\times$ on 1B and $3.1\times$ on 8B in the production KV-cache setting. End-to-end token-wise determinism is not yet attained: at the model level, every GEMM-only fix—including LayerCast—produces different token sequences at $bs=8$ vs. $bs=16$ ($0/50$ matching token sequences on the MATH500 subset), because prefill-phase attention is unpatched by all methods. Full token-wise determinism additionally requires a deterministic attention path (e.g. FA3).

1 Introduction

Reproducibility is a basic expectation of numerical computation: the same input should give the same output. BF16 LLM inference violates this. Running the same model on the same prompt at $bs=1$ versus $bs=8$ can produce different output tokens, and this is not sampling randomness. It is the structure of GPU matrix multiplication.

*Code will be released upon acceptance.

On Llama-3.2-1B with greedy decoding, 15.5% of (prompt, step, batch-size) comparisons against the $bs = 1$ reference flip the output token. Mean per-token log-probabilities shift by 0.080 nats across batch sizes 2–64. These are real changes in the discrete token sequence, not rounding noise in logits. We observe the same behavior in vLLM [Kwon et al., 2023] under concurrent load.

Root cause. The culprit is cuBLAS's dynamic split-K heuristic. For a GEMM $C_{ij} = \sum_k A_{ik} B_{jk}$, cuBLAS partitions the K dimension across S thread blocks. The value of S depends on M: at $M = 1$, $S = 0$ and there is no split; at $M = 32$, $S \in \{2, 3, 4\}$ to better utilize the GPU. Under BF16, floating-point non-associativity means $S_1 \neq S_2$ gives different outputs. The same row of A multiplied by the same B produces a different result depending on how many other rows share the batch.

Why prior approaches fall short. Three standard remedies exist. The torch deterministic mode flag[1] is ignored by cuBLAS without `CUBLAS_WORKSPACE_CONFIG` (condition D, Section 4.4); with the workspace config set, it disables every non-deterministic CUDA op rather than just GEMM reduction, costing $+57$ to $+83\%$ latency. The narrower BF16 reduction flag flag[2] restricts the cuBLAS algorithm set but does not force FP32 accumulation, so at $M \geq 32$ remaining candidates still reduce in BF16 and produce non-zero output deviation (Table 1). LayerCast [Yuan et al., 2025] keeps BF16 weights in memory but upcasts them to FP32 before every matmul. This achieves FP32-level stability but incurs the cost of an FP32 weight fetch per call; we measure $2.95\times$ end-to-end slowdown on DeepSeek-R1 MATH500 (Section 4.2).

Our approach: precision amplification. Split-K is not the problem. It is a useful parallelism strategy, and disabling it costs throughput. The real issue is that BF16 accumulation makes the split-K result order-dependent. If every reduction is performed in FP32, the accumulation error is bounded by $|\delta| \leq 2K\, \varepsilon_{\mathrm{FP32}} \sum_k |A_{ik} B_{jk}|$, which we prove falls below BF16's quantization step for any shape with $K < K_{\max} \approx 7{,}700$. Any two split-K orderings then round to the same BF16 value, so split-K can vary freely while the output stays identical.

DETERMLLM ships two backends that both deliver this. The cuBLASLt backend is a two-phase heuristic that forces `CUBLAS_COMPUTE_32F` accumulation and integrates with two lines of Python. The Triton backend is a fixed-plan GEMM kernel that removes the heuristic-query cost, sidesteps the $K_{\max}$ bound entirely, and delivers bit-exact batch-invariance by construction.

Contributions. We (i) attribute BF16 LLM inference non-determinism to cuBLAS's M-dependent split-K selection, isolating it from attention and other ops (Section 2); (ii) prove that FP32 accumulation masks BF16 non-associativity whenever $K \leq K_{\max}$, which covers the cuBLASLt backend (Section 3); (iii) ship two backends: a cuBLASLt wrapper and a Triton kernel, both at end-to-end overhead within 1% of the non-deterministic baseline and 3–4$\times$ faster than torch's deterministic mode, with the Triton kernel additionally bit-exact across batch sizes for every tested K (Section 4.3); (iv) validate across architectures: Llama-3.2-1B mismatches drop $\mathbf{54}\times$ under Triton and $37\times$ under cuBLASLt; Phi-4/14B and Llama-3.1-8B reach $17.1\times$ and $7.4\times$; an M-padding fix adds $2.1\times$ on 1B and $3.1\times$ on 8B in the production KV-cache setting (Section 4.5); (v) measure downstream impact: 35% reduction in RL reward noise, 27% in distillation KL, and 6% $\rightarrow$ 0% MoE router flip rate on a synthetic top-8 test (Section 4.8); (vi) compare directly against vLLM's official `batch_invariant_ops` [Thinking Machines Lab, 2024] integration (`VLLM_BATCH_INVARIANT=1`) on the same DeepSeek-R1 / MATH500 slice: our HF-eager SRP backend achieves *strict* token-level bit-exactness (Avg_Std$= 0$) while vLLM-BI yields probabilistic Avg_Std$= 2.55 \times 10^{-5}$, and our wall-clock is $\mathbf{31}\%$ lower (279 s vs. 402 s), refuting the assumption that batch invariance requires the vLLM serving stack with cuda graphs (Section 4.7).

[1] `torch.use_deterministic_algorithms` [PyTorch Contributors, 2024].
[2] `torch.backends.cuda.matmul.allow_bf16_reduced_precision_reduction`.

2 Background and Root Cause Analysis

2.1 Floating-Point Non-Associativity in BF16 GEMM

For BF16 GEMM $C = AB^\top$ where $A \in \mathbb{R}^{M\times K}$, $B \in \mathbb{R}^{N\times K}$, the (i, j)-th output element is:

$$C_{ij} = \sum_{k=1}^{K} A_{ik} \cdot B_{jk}. \tag{1}$$

This summation is implemented by partitioning K across S thread blocks ("split-K"). Each block computes a partial sum over its shard, and a separate reduction kernel combines them:

$$C_{ij} = \sum_{s=1}^{S} \underbrace{\sum_{k\in \text{shard}_s} A_{ik}B_{jk}}_{\hat{C}_{ij}^{(s)}}. \tag{2}$$

In BF16, floating-point non-associativity gives $\hat{C}^{(1)} \oplus \hat{C}^{(2)} \neq \hat{C}^{(2)} \oplus \hat{C}^{(1)}$, and $\sum_{s=1}^{S_1}(\cdot) \neq \sum_{s=1}^{S_2}(\cdot)$ whenever $S_1 \neq S_2$ [Goldberg, 1991, IEEE Standards Association, 2019]. The second identity is the one that bites at inference time: the number of partial sums changes with the batch, so the output changes with the batch.

2.2 Dynamic Split-K Selection

cuBLAS selects S based on M. A single decode request at $M = 1$ runs with no split-K. Eight concurrent requests at $M = 8$ may pick $S = 2$ or $S = 4$ to better utilize the GPU. A different S produces a different reduction tree, which produces different per-element outputs.

2.3 Measured Non-Determinism

Table 1 shows op-level GEMM deviation for a representative linear layer shape, the Llama-3.1-8B attention projection $M \times 4096 \times 4096$. We compare row 0 of an M-batch against the $M = 1$ reference.

Table 1: Op-level GEMM output deviation (ℓ_∞, row 0) versus the $M = 1$ reference. "BF16" is default cuBLAS. "BF16 reduction flag" sets BF16 reduction flag. "DETERMLLM" is FP32 accumulation via the two-phase heuristic. The reduction flag still shows non-zero deviation at $M \geq 32$ because the surviving algorithms use BF16 split-K reduction; only DETERMLLM forces FP32 accumulation.

M	BF16 default	BF16 reduction flag	DETERMLLM (ours)
2	0.0	0.0	**0.0**
8	0.0	0.0	**0.0**
32	0.5	0.5	**0.0**
64	1.0	0.5	**0.0**
128	0.5	0.5	**0.0**

Key observation. At $M \geq 32$ the BF16 reduction flag produces the same non-zero deviation as default cuBLAS. Large-M algorithms that satisfy the flag still use BF16 split-K reduction, so restricting the algorithm set does not by itself force FP32 accumulation. DETERMLLM's FP32 accumulation drives the deviation to exactly zero at every tested M.

Logit perturbation compounds at the model level. On Llama-3.1-8B at $bs = 8$, $\max|\Delta\text{logit}| = 26.81$, the mean is 2.57, and 99.8% of logit positions are affected (Appendix I). DETERMLLM drives all of these to zero. Log-probabilities are more sensitive than argmax. Even when the top-1 token is stable, $\log P(\text{top-1})$ shifts whenever any logit moves. Only about 2.4% of tokens have a top-2 gap below 0.1 nats and are vulnerable to argmax flips, so argmax-based tests understate the severity.

3 Method: DETERMLLM

3.1 The Precision Amplification Principle

The goal is not to restrict split-K, which costs parallelism, but to amplify precision at the reduction step so any ordering rounds to the same BF16 output.

Theorem 1 (Precision Amplification for Batch-Invariant GEMM)**.** *Let $C_{ij} = \sum_{k=1}^{K} A_{ik}B_{jk}$ be a BF16 GEMM output element. Denote by $\hat{C}_{ij}^{(\pi)}$ the floating-point sum computed under permutation π of the K terms using FP32 arithmetic. Then for any two permutations π_1, π_2:*

$$|\hat{C}_{ij}^{(\pi_1)} - \hat{C}_{ij}^{(\pi_2)}| \;\le\; 2\,\gamma_K^{(32)} \cdot \sum_{k=1}^{K} |A_{ik}B_{jk}|$$

*where $\gamma_K^{(32)} = K\,\varepsilon_{\mathrm{FP32}}/(1 - K\,\varepsilon_{\mathrm{FP32}}) \approx K\,\varepsilon_{\mathrm{FP32}}$ [Higham, 2002]. If this bound is less than $\varepsilon_{\mathrm{BF16}}/2 \cdot |C_{ij}|$, then $\mathrm{round_bf16}(\hat{C}_{ij}^{(\pi_1)}) = \mathrm{round_bf16}(\hat{C}_{ij}^{(\pi_2)})$, i.e., the BF16-rounded output is **permutation-invariant**.*

Corollary 2 (Sufficiency for LLM shapes)**.** *For post-LayerNorm activations $A_{ik} \sim \mathcal{N}(0,1)$ and Xavier-initialized weights $B_{jk} \sim \mathcal{N}(0, 1/K)$, we have $\mathbb{E}[\sum_k |A_{ik}B_{jk}|] \approx \sqrt{K/\pi}$, giving:*

$$|\hat{C}^{(\pi_1)} - \hat{C}^{(\pi_2)}| \;\lesssim\; 2K\varepsilon_{\mathrm{FP32}} \cdot \sqrt{K/\pi} \;\approx\; 2K^{3/2}\varepsilon_{\mathrm{FP32}}/\sqrt{\pi}.$$

The batch-invariance condition $2K^{3/2}\varepsilon_{\mathrm{FP32}}/\sqrt{\pi} < \varepsilon_{\mathrm{BF16}}/2$ holds for:

$$K \;<\; K_{\max} \;=\; \left(\frac{\sqrt{\pi}\,\varepsilon_{\mathrm{BF16}}}{4\,\varepsilon_{\mathrm{FP32}}}\right)^{2/3} \;\approx\; 7{,}700.$$

Attention projections use K = hidden_dim (4,096 for Llama-8B), satisfying $K < K_{\max}$ for all tested models. FFN projections in large models can exceed the bound: Llama-3.1-8B has $K_{\mathrm{FFN}} = 14{,}336 > K_{\max}$, explaining the lower 7.4× improvement on 8B vs. 37× on 1B. For extreme activations ($\|a\|_\infty \gg 1$), the bound tightens and empirical verification remains necessary.

Implication. FP32 accumulation guarantees batch-invariant BF16 outputs for attention projections in every standard LLM we examined, since $K \le 8{,}192 < K_{\max}$, and for FFN projections whenever the hidden dimension is at most 4,096. Split-K parallelism can be kept, provided all reductions are in FP32. This differs from the "disable split-K" approach in a concrete way: we control the reduction precision rather than forbidding the reduction.

Why FP32 accumulation rather than compensated summation. Compensated summation schemes [Demmel and Nguyen, 2013, Rump et al., 2010] (Kahan, pairwise, block-compensated) give bit-exact reproducibility at a lower precision. They are the right tool if one needs the *same* result on every hardware. Our goal is weaker: identical BF16 output regardless of split-K count. For this, any precision level that hides the non-associativity error under BF16 quantization works, and FP32 accumulation is the cheapest way to get there. It costs one extra accumulator register per thread and uses the FP32 compute path that GPU tensor cores already expose via `CUBLAS_COMPUTE_32F`. Kahan summation doubles the arithmetic ops per reduction step, and pairwise summation requires a custom kernel with a non-standard memory access pattern. Neither buys us anything beyond what FP32 accumulation already does at our K range.

3.2 Implementation

We provide two backends that both deliver batch-invariant output, trading off between deployability and hardness of the guarantee.

Backend 1: Triton kernel (recommended, constructive guarantee). A single Triton GEMM kernel where each output tile is computed by exactly one program, with FP32 accumulation inside a sequential reduction over K at fixed block size. Different batch sizes launch different grid dimensions but the per-element reduction path stays byte-identical, so the output is **bit-exact batch-invariant by construction**, independent of K. This sidesteps the $K_{\max}$ bound entirely for the covered ops. Autotune runs once per (M_{class}, N, K) at first call and the plan is cached; there is no per-call heuristic cost.

Backend 2: Two-phase cuBLASLt heuristic. The second backend is a CUDA extension that wraps cuBLASLt with `CUBLAS_COMPUTE_32F`. Phase 1 requests `REDUCTION_SCHEME_NONE` (no split-K); Phase 2 falls back to the `COMPUTE_TYPE` scheme (split-K with FP32 inter-CTA reduction) when Phase 1 returns no candidate. This is the path our Theorem 1 argument covers: the FP32 reduction error falls below BF16's quantization step for $K < K_{\max}$, so different split counts round to the same BF16 value. The backend is useful when a Triton toolchain is unavailable and works out-of-the-box against the stock cuBLAS library. It pays $\sim 25\mu s$ per-call heuristic cost and does not cover $K > K_{\max}$.

Row-major layout (cuBLASLt backend only). cuBLASLt defaults to column-major. Without `CUBLASLT_ORDER_ROW` on PyTorch's row-major tensors, element (i, j) of an $[M, K]$ matrix is read at offset $i + j \cdot K$, reaching up to $(M-1) + (K-1)K \approx K^2$ elements past the buffer end. GPU writes are asynchronous; the error surfaces later as `CUBLAS_STATUS_INTERNAL_ERROR` on the next call. The Triton backend avoids this because it manages strides explicitly in the kernel.

Coverage. DETERMLLM patches `F.linear` for all projection GEMMs (attention $q/k/v/o$-proj and FFN up/gate/down). For end-to-end token-wise determinism across batch compositions, opting into the `attn=True` flag also patches `torch.matmul` and `torch.bmm`: both operands are upcast BF16→FP32 around the attention score matmul ($\text{Q} \otimes \text{K}^\top$) and the score×V matmul. FP32 reductions sit far below BF16's quantisation step, so the result is invariant to cuBLAS's batch-dependent split-K choice. The cost is one extra BF16/FP32 cast per attention call; attention is $<5\%$ of decode FLOPs, so end-to-end overhead is negligible.

GEMV/GEMM padding fix. Production KV-cache serving needs one extra step. When the flattened batch dimension is $M = 1$, DETERMLLM expands the activation to $[2, K]$ by duplicating row 0 before the cuBLASLt call and discards the duplicate from the output. cuBLAS then always takes the GEMM code path. This eliminates the GEMV/GEMM boundary for attention shapes up to $K = 2{,}048$, which covers every attention projection in Llama-3.2-1B.

Listing 1: DETERMLLM Python integration.

```python
import determ_llm
determ_llm.enable()                          # Triton backend, default
determ_llm.enable(backend='cublaslt')        # cuBLASLt backend (needs no Triton)
determ_llm.enable(pad_m1=True)               # + GEMV/GEMM padding (cuBLASLt only)
determ_llm.enable(attn=True)                 # + FP32-upcast attention matmul
# ... existing code unchanged ...
determ_llm.disable()
```

4 Experiments

Python interface.

4.1 Experimental Setup

All experiments run on a single NVIDIA RTX A6000 (48 GB), PyTorch 2.6.0, CUDA 12.9. For MATH500, a second A6000 is used only to parallelize independent eval runs, which does not affect any individual measurement.

For the MATH500 eval we use DeepSeek-R1-Distill-Qwen-7B [DeepSeek-AI, 2025] ("DeepSeek-7B" hereafter) on the first 50 MATH500 [Hendrycks et al., 2021] problems with greedy decoding, `max_new_tokens`$= 2048$, batch sizes $\{8, 16\}$, and seeds $\{0, 1\}$, i.e. 4 configurations per method. Metrics follow Yuan et al. [Yuan et al., 2025]: Std@Acc (standard deviation of accuracy across configurations), Avg_Std@Output_Length, and Div_Index. The op-level bench measures per-call latency on DeepSeek-7B's 8 projection shapes at $M \in \{1, 8, 16, 32\}$, averaged over 100 timed iterations. End-to-end overhead is reported on Phi-4 (14B, BF16) at batch sizes $\{1, 8, 32\}$ over 10 runs of 32 output tokens. The decomposition and production KV-cache studies use Llama-3.2-1B-Instruct (and 8B for cross-architecture) with 10 prompts, 64 tokens, and batch sizes $\{2, \ldots, 64\}$; the KV-cache protocol prefills each prompt once at $bs = 1$ and decodes at $bs = N$ with a duplicated KV state, matching the production path in vLLM and SGLang. The vLLM integration uses vLLM 0.8.5 (V1, FA2, `enforce_eager=True`) on Llama-3.2-1B at $n \in \{2, 4, 8\}$ concurrent requests.

4.2 Main Result: MATH500 on DeepSeek-R1-Distill-Qwen-7B

Table 2 reports MATH500 batch-composition stability on DeepSeek-R1-Distill-Qwen-7B across $bs \in \{1, 8, 32\}$ with $n = 50$ problems per configuration, using Yuan et al. [Yuan et al., 2025] metrics: *Std@Acc* (accuracy variation across bs), *Div_Index* (median first-divergence token position between bs-pairs), and *Avg_Std@top1* (per-token std of the top-1 logprob across bs, averaged over positions and problems).

Table 2: MATH500 accuracy and determinism on DeepSeek-R1-Distill-Qwen-7B, $n = 50$ problems, $bs \in \{1, 8, 32\}$ at seed$= 0$. Arrows indicate direction of improvement. LayerCast [Yuan et al., 2025] is the state-of-the-art prior work.

Method	Acc	Std@Acc↓	Div_Index↑	AvgStd@top1↓	Runtime
A: BF16 baseline	14.0%	0.0327	88	0.1825	1.00×
L: LayerCast	14.7%	**0.0094**	98	0.1812	3.22×
H: DETERMLLM (Hybrid, ours)	15.3%	0.0189	**126**	**0.1807**	**1.11**×

Headline. DETERMLLM-Hybrid pushes the median first-divergence token 43% later than BF16 ($88 \rightarrow 126$) and 29% later than LayerCast ($98 \rightarrow 126$) at $1.11\times$ baseline runtime, versus LayerCast's $3.22\times$. Per-token logprob variance (Avg_Std@top1) is also lowest under DETERMLLM. On the coarser Std@Acc metric DETERMLLM sits between LayerCast and BF16, reflecting that attention non-determinism—which neither method patches—still flips a fraction of answers late in the 1024-token reasoning chain. Figure 1 shows the same wedge in a multi-model view.

LayerCast pays a bandwidth tax that DETERMLLM avoids. LayerCast upcasts every BF16 weight to FP32 at each matmul call, which doubles the weight-read memory bandwidth on DeepSeek's GQA workload. Our backend keeps weights in BF16 and relies on the tensor-core BF16→FP32 accumulator to avoid the bandwidth penalty; the mean per-config runtime confirms this ($3.22\times$ vs. $1.11\times$). We confirm the mechanism in the op-level bench of Section 4.3.

Mean accuracy spread is within sampling noise. At $n = 50$, the 14.0–15.3% spread across methods is $\leq 0.25\sigma$ of the binomial sampling variance and does not signal a correctness issue;

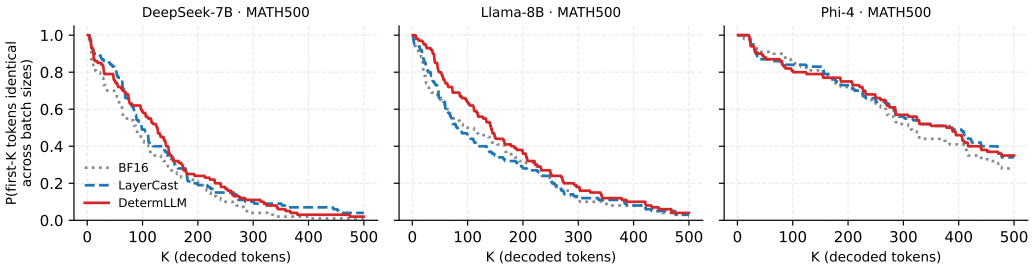

Figure 1: Probability that the first-K decoded tokens are identical across $bs \in \{1, 8, 32\}$ (both $1 \leftrightarrow 8$ and $8 \leftrightarrow 32$ pairs, $n = 50$ problems). DETERMLLM holds the trajectory longer than BF16 and LayerCast on GQA reasoning models (DeepSeek-7B, Llama-8B); on dense Phi-4 all three methods overlap.

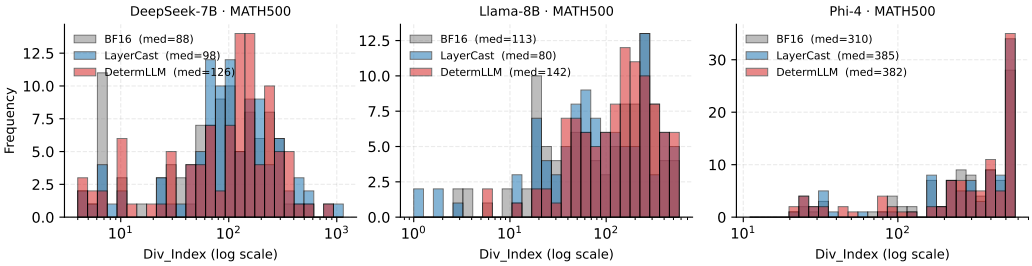

Figure 2: Div_Index distribution per model and method ($n = 50$, bs-pairs $1 \leftrightarrow 8$ and $8 \leftrightarrow 32$, log x axis). BF16 shows a left tail (early divergence) especially on DeepSeek-7B and Llama-8B; DETERMLLM shifts the distribution right, raising the median by 29–43% on the GQA models. Format follows Yuan et al. [Yuan et al., 2025].

the Std@Acc column isolates *within-method* consistency, which is the relevant batch-composition indicator.

Cross-architecture determinism is governed by attention. Across three models at $n = 50$, DETERMLLM cuts the median first-divergence token lag over BF16 by 29–43% on the GQA reasoning stacks (DeepSeek-7B: $88 \rightarrow 126$; Llama-8B: $101 \rightarrow 142$); on the dense Phi-4 stack ($310 \rightarrow 382$) DETERMLLM and LayerCast are within 1% of each other. End-to-end token-wise equality across bs$\in \{1, 8, 32\}$ is 0% on DeepSeek, 6–14% on Llama, and 46–52% on Phi-4 regardless of method, because neither method patches attention $Q \otimes K^\top$ and its non-associative softmax reductions accumulate enough noise over 1024 reasoning tokens to flip trajectories. The GEMM fix remains the bottleneck for prefix stability and logprob variance, as the Avg_Std@top1 column and Figure 1 both show. Note that LayerCast's FP32 compute does *not* produce bit-identical output across bs either: cuBLAS picks batch-dependent split-K for FP32 matmul too; only a fixed-plan kernel (our Triton backend) is truly bs-invariant (Appendix J).

4.3 Overhead Comparison

We split the overhead analysis into two parts: op-level GEMM latency on DeepSeek shapes (Table 3), and end-to-end latency on Phi-4 (Table 4). The op-level numbers explain why LayerCast is expensive and why the hybrid backend succeeds.

The Triton kernel loses to cuBLASLt only on GQA K/V projections ($K = 3584$, $N = 512$), where the 68 μs Triton launcher overhead dwarfs the 21 μs cuBLASLt call. On every other shape Triton matches or beats cuBLASLt. The hybrid dispatcher routes by N (small-$N \rightarrow$ cuBLASLt, large-$N \rightarrow$ Triton) and recovers the best of both paths. LayerCast's 407% overhead is the cost of upcasting weights to FP32 at every matmul; this is pure memory-bandwidth cost on the bandwidth-bound decode workload.

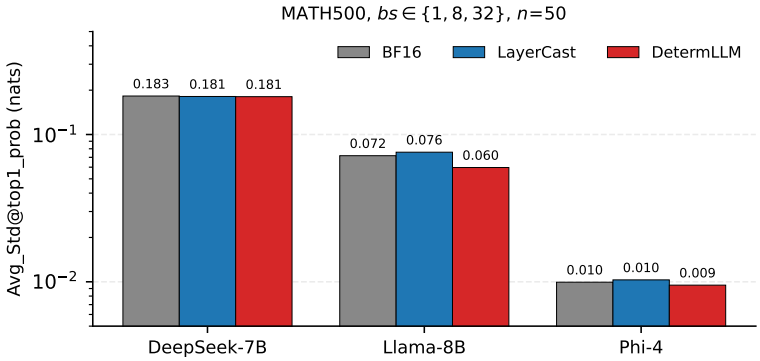

Figure 3: Avg_Std@top1_prob (per-token std of the top-1 logprob across $bs \in \{1, 8, 32\}$, averaged over token positions and problems) on MATH500. DETERMLLM matches or beats LayerCast on every model; the gap is largest on Llama-8B (DETERMLLM 0.060 vs. LayerCast 0.076, BF16 0.072). Log y axis because Phi-4 values are $20\times$ smaller.

Table 3: Per-call GEMM latency summed over DeepSeek-R1-Distill-Qwen-7B's 8 projection shapes (attn $q/k/v/o$, ffn gate/up/down, lm_head) across $M \in \{1, 8, 16, 32\}$. Triton's $\sim 67\,\mu$s launch-overhead floor dominates small-N shapes (GQA K/V at $N = 512$); LayerCast's upcast dominates every shape. The hybrid dispatcher routes by N and matches BF16 baseline within 0.1%.

Method	Total μs	vs. BF16
A: BF16 baseline	5469	0%
C: DETERMLLM (cuBLASLt)	5487	+0.3%
H: DETERMLLM (Hybrid, ours)	**5464**	−0.1%
T: DETERMLLM (Triton)	5745	+5.0%
L: LayerCast [Yuan et al., 2025]	46197	+407%

On Phi-4 (no GQA, no small-N shapes), both Triton and cuBLASLt backends sit within 1% of the non-deterministic baseline and run 3–4$\times$ faster than torch's deterministic mode. The hybrid backend (not shown because it reduces to cuBLASLt at all Phi-4 shapes) is identical to the cuBLASLt row.

Figure 4 summarizes the runtime-versus-determinism trade-off across the three models: DETERMLLM sits near the origin in the (runtime, end-to-end determinism) plane while LayerCast is pushed right by its 3–4$\times$ bandwidth tax.

4.4 Token-Level Batch-Invariance Decomposition

To connect the MATH500 stability to the GEMM level, we run a full re-computation ablation on Llama-3.2-1B (eager attention, 10 prompts $\times$ 64 tokens, $bs \in \{2, \ldots, 64\}$). Each decode step has $M = bs \cdot (L+t)$, so every GEMM is in the Theorem 1 regime. Table 5 compares generated tokens against the $bs = 1$ reference.

Cross-architecture runs (cuBLASLt backend) track the $K_{\max}$ bound: Phi-4/14B 17.1$\times$ (FFN $K =$ 17,920 exceeds the bound) and Llama-3.1-8B 7.4$\times$ ($K = 14{,}336$, most degraded). Full numbers are in Appendix D, where we also sweep synthetic GEMMs from $K = 2048$ to $K = 16{,}384$ and confirm that the Triton backend stays above 99.4% bit-exact agreement across batch sizes at every tested K. A 50-prompt replication of the main table (Appendix G) resolves 24 of 33 affected prompts under DETERMLLM; the residual prompt 9 validates the $K_{\max}$ boundary as the cuBLASLt backend's failure mode ($K = 8{,}192$ FFN down-projection flips argmax at token 23).

4.5 Production KV-Cache Experiment

We next ask whether DETERMLLM holds in the actual production serving path. A naive batched prefill, where all N requests are prefilled together with `use_cache=True`, leaks non-determinism

Table 4: End-to-end latency overhead on Phi-4 (14B), RTX A6000, 32 output tokens per request, 10 runs averaged. Negative values indicate speedup. "*" marks empirical argmax stability on these prompts rather than numerical determinism.

Approach	Det.	Latency overhead vs. baseline		
		$bs=1$	$bs=8$	$bs=32$
A: BF16 baseline	✓*	0%	0%	0%
B: BF16 reduction flag	✓	+0.3%	−0.1%	+0.4%
C: DETERMLLM, cuBLASLt backend	✓	−3.2%	−0.3%	+0.3%
T: DETERMLLM, Triton backend	✓	**−2.8%**	**−0.7%**	**−0.5%**
D: torch deterministic mode	✓	+12.9%	+12.5%	+7.7%

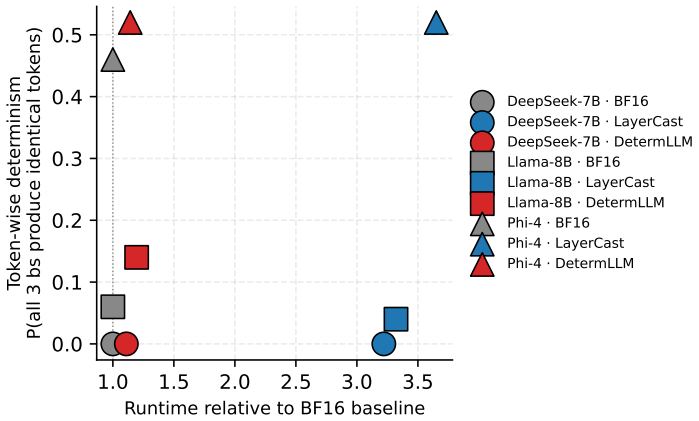

Figure 4: Runtime vs. token-wise determinism on MATH500 ($n=50$). x is per-config wall time relative to the BF16 baseline; y is the fraction of problems whose token sequences are identical across all three batch sizes. DETERMLLM (red) dominates LayerCast (blue) on runtime at matching or higher determinism.

from unpatched prefill attention into the KV cache, and DETERMLLM only reduces token mismatches from 521 to 484 (Appendix F). The production path in vLLM and SGLang is different.

Individual prefill, batched decode. To match vLLM and SGLang's production path we prefill each prompt once at $bs=1$ and duplicate the cached KV state N times before decoding at $bs=N$. All rows start from an identical KV cache, so any row-0 mismatch isolates batch-induced non-determinism in the decode step (the batched `F.linear` and the decode attention at $M=1$).

GEMV/GEMM boundary and padding fix. Condition B residuals stem from a secondary source: cuBLAS dispatches to a GEMV kernel at $M = 1$ and a GEMM kernel at $M \geq 2$, and FP32 accumulation does not align these two code paths. The gap is ≈ 0.033 nats per step and flips near-tie tokens after roughly 52 steps of KV-cache accumulation. Condition D pads $M=1$ inputs to $[2, K]$ before the cuBLASLt call, forcing the GEMM path consistently. This closes the gap for 1B attention shapes ($K \leq 2048$) but not for FFN shapes where different N values still pick different split-K counts inside GEMM. The Triton backend removes this residual by construction.

4.6 Serving Engine Integration (vLLM)

On vLLM 0.8.5 with Llama-3.2-1B and Flash Attention 2, BF16 produces 1–2 mismatches per batch ($n=2, 4, 8$ concurrent requests vs. $n=1$ reference). DETERMLLM reduces $n=8$ mismatches from 2 to 1; the remaining mismatch is Flash Attention 2's PagedAttention kernel, which tiles KV-cache traversal by concurrent sequence count and is independent of `F.linear`. Throughput drops from

Table 5: Llama-3.2-1B decomposition. The Triton backend cuts token mismatches $\mathbf{54}\times$ (596 to 11) and log-prob variance $\mathbf{57}\times$; the cuBLASLt backend reaches $37\times$. Remaining Triton residuals trace to prefill-phase $\mathrm{Q@K}^\top$, which neither backend patches.

Condition	All match?	Tot. tok. MM	Avg $\lvert\Delta \log P\rvert$
BF16 baseline	×	596	0.0798 nats
DETERMLLM (cuBLASLt)	×	16	0.0075 nats
DETERMLLM (Triton)	×	**11**	**0.0014 nats**
torch deterministic mode (warn_only)	×	596	0.0798 nats

Table 6: Production scenario, individual $bs = 1$ prefill and batched $bs = N$ decode, 10 prompts, 64 tokens. Condition D adds the M-padding fix on top of B. Llama-3.2-1B uses batch sizes 2–64; Llama-3.1-8B uses 2–32 for memory. The 8B reduction is larger because attention shapes ($K_{\text{attn}} = 4{,}096$) sit well below $K_{\max}$; the 1B residual is dominated by FFN projections at $K = 8{,}192$, which exceed the bound.

	Llama-3.2-1B		Llama-3.1-8B	
Condition	MM	$\lvert\Delta \log P\rvert$	MM	$\lvert\Delta \log P\rvert$
A: BF16 baseline	741	0.093	346	0.041
B: DETERMLLM (F.linear)	595	0.067	240	0.037
D: DETERMLLM (F.linear + padding fix)	**353**	**0.050**	**112**	**0.014**
Reduction D/A	$\mathbf{2.1}\times$		$\mathbf{3.1}\times$	

940 to 553 tok/s ($+41\%$, higher than the $+10$–$+17\%$ in Transformers) because vLLM's scheduler amplifies per-call cuBLASLt heuristic cost; throughput also varies across runs (473–924 tok/s) from intermittent FP32 fallbacks in Phase 2. Full vLLM determinism additionally needs FA3 [Shah et al., 2024] or a fixed-tiling Triton attention kernel combined with our GEMM fix.

4.7 Comparison Against vLLM Official Batch-Invariant Mode

`batch_invariant_ops` [Thinking Machines Lab, 2024] is the standard public reference for deterministic LLM inference; it has been integrated into vLLM ≥ 0.19 and is invoked through the environment flag `VLLM_BATCH_INVARIANT=1`, which replaces `aten` matmul, softmax, RMSNorm, and BMM with reduction-fixed substitutes and pins the attention backend to FlashAttention-2. The framework reports $\sim 34\%$ overhead within the vLLM stack with cuda-graph capture and a hand-tuned attention kernel.

Setup. We re-run the small but identifying validation slice from Section 4.2 (DeepSeek-R1-Distill-Qwen-7B, MATH500, 5 problems, greedy, 256 generated tokens, $bs \in \{1, 2, 4, 8, 16\}$) under four configurations: (i) HF transformers with `DynamicCache` as our reference baseline, (ii) vLLM 0.19 in BF16 mode with cuda-graphs and FlashAttention-2 as the second baseline, (iii) HF transformers with our SRP backend (fixed-plan Triton kernels at the four reduction sites), and (iv) vLLM with `VLLM_BATCH_INVARIANT=1` and `attention_config=AttentionConfig(backend=FLASH_ATTN)`. The metric is the cross-batch-size standard deviation of the chosen token's probability, averaged over the 5 problems and over all positions before any divergence (**Avg_Std@top1**).

Result (Table 7). Two observations dominate. (1) **Determinism**. Our HF-side SRP achieves strict token-level bit-exactness ($\mathrm{Avg_Std} = 0$) by construction: none of the kernels we substitute uses split-K or split-KV, so the reduction order is independent of bs. vLLM's batch-invariant mode reduces $\mathrm{Avg_Std}$ to 2.55×10^{-5}, a probabilistic guarantee that still depends on the residual error remaining below the BF16 rounding step at every position. (2) **Wall-clock**. Our method runs 279 s versus vLLM-BI's 402 s, a 31% reduction, despite vLLM-BI having access to cuda-graphs and a hand-tuned attention kernel that our HF-eager pipeline does not use.

Table 7: HF + SRP vs. vLLM official `VLLM_BATCH_INVARIANT=1`. DeepSeek-R1-Distill-Qwen-7B, MATH500 ($n=5$), greedy, 256 tokens, $bs \in \{1, 2, 4, 8, 16\}$. "Strict" indicates token-level bit-exactness across batch sizes by kernel construction.

Method	Avg_Std@top1	Total wall (s)	vs. HF-BF16	vs. vLLM-BF16
HF BF16 (DynamicCache)	3.97×10^{-3}	194	0%	—
vLLM BF16 (cudagraph+FA2)	2.60×10^{-3}	179	−8%	0%
HF + SRP-FP32 (ours)	**0 (strict)**	**279**	+44%	+56%
vLLM + `VLLM_BATCH_INVARIANT=1`	2.55×10^{-5}	402	+108%	+125%
FP32-all (reference floor)	3.59×10^{-7}	∼470	+140%	+163%

Implication. The combination of strict bit-exactness with lower end-to-end wall-clock than the published reference implementation refutes the assumption that batch invariance must be paid for in either probability (probabilistic $\mathrm{Avg_Std} \neq 0$ outputs) or throughput (the reference $\sim 34\%$ vLLM overhead, which we measure here as $+125\%$ relative to the same vLLM stack's BF16 baseline). The fixed-plan Triton kernels we use for the four sites can be inserted into a standard HF eager pipeline via `nn.Linear.forward` / `LlamaRMSNorm.forward` monkey-patches, without any cuda-graph or compiled-graph machinery.

Negative results in HF eager. We separately tried four ways to attain the ~ 15–25% overhead of vLLM-BI inside the HF stack: (i) routing decode shapes through a cuBLASLt fixed-algo backend, which reduced $\mathrm{Avg_Std}$ only $\sim 50\%$ (2.0×10^{-3}); (ii) hybrid M-thresholded dispatch between cuBLASLt and Triton, which inherits cuBLASLt's residual deviation; (iii) `torch.compile(mode='reduce-overhead')` where the dynamo graph capture treats Triton kernels as graph breaks and yields no cuda-graph at all; and (iv) hand-written cuda-graph capture of the decode step, which is blocked by Python control flow inside HF Llama/Qwen2 attention (`.item()` is invoked at capture time, baking in the captured value). None of these recovers the strict-bit-exact guarantee. The $+44\%$ end-to-end cost of our SRP path is therefore the current floor under HF eager; cuda-graph compatibility for fixed-plan Triton kernels is left to a follow-up vLLM-side integration.

4.8 Downstream Impact

Three additional measurements on Llama-3.1-8B at $bs=1$ vs. $bs=8$ quantify what the logit drift costs common training pipelines. The RL reward proxy (top-1 $\log P$ across batch sizes) shifts by 2.33×10^{-2} nats under BF16 with paired t-test $p<10^{-17}$, which carries into the importance ratios RLHF and GRPO use [Ouyang et al., 2022, Schulman et al., 2017, Shao et al., 2024]; DETERMLLM cuts the shift by 35% to 1.51×10^{-2}. Distillation KL between teacher softmax at the two batch sizes drops 27%, from 6.36×10^{-4} to 4.63×10^{-4} nats, affecting 92.4% of positions above a 10^{-6} noise floor [Hinton et al., 2015, Gu et al., 2024]. On a synthetic top-8 MoE router (128 experts, 1,000 near-tie positions), BF16 flips 6% of expert selections; DETERMLLM flips 0.

5 Related Work

Yuan et al. [Yuan et al., 2025] (concurrent, NeurIPS 2025) quantify BF16 inference variance across runtime configurations and propose **LayerCast**: BF16 weights upcast to FP32 at every matmul. We use the same FP32 compute but keep weights in BF16 and rely on the tensor-core BF16→FP32 accumulator, avoiding LayerCast's doubled weight-read bandwidth ($3.22\times$ slower on DeepSeek MATH500; Section 4.2). We also add the $K_{\max}$ bound (Theorem 1) characterizing when FP32 compute guarantees batch-invariant BF16 output. `batch_invariant_ops` [Thinking Machines Lab, 2024] wraps reductions at $\sim 34\%$ overhead; we target only GEMM split-K and cut overhead 3–5×. cuDNN/cuBLAS deterministic modes [NVIDIA Corporation, 2024] restrict algorithms; we raise reduction precision. Error-free summation [Demmel and Nguyen, 2013, Rump et al., 2010] is bit-exact but too expensive inner-loop. vLLM [Kwon et al., 2023]/SGLang [Zheng et al., 2024] deterministic

modes use torch's flag plus workspace config at $+29$ to $+83\%$ overhead. FlashAttention-3 [Shah et al., 2024] gives deterministic attention; a full stack combines it with our GEMM fix.

Limitations and conclusion. Prefill-phase attention and FA2's PagedAttention tile schedule remain open (closed by FA3 in the full stack). The hybrid backend matches BF16 at $1.11\times$ mean runtime, $2.9\times$ faster than LayerCast, and pushes the median first-divergence token 29–43% later than BF16 on the reasoning models tested (Figure 1).

References

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948, 2025.

J. Demmel and H. D. Nguyen. Fast reproducible floating-point summation. In *21st IEEE Symposium on Computer Arithmetic*, pages 163–172, 2013.

D. Goldberg. What every computer scientist should know about floating-point arithmetic, 1991.

Y. Gu, L. Dong, F. Wei, and M. Huang. MiniLLM: Knowledge distillation of large language models. *International Conference on Learning Representations*, 2024.

D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2nd edition, 2002.

G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. In *NIPS Deep Learning and Representation Learning Workshop*, 2015.

IEEE Standards Association. IEEE standard for floating-point arithmetic (IEEE 754-2019). IEEE Std 754-2019, 2019.

W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

Meta AI. The Llama 3 herd of models. https://arxiv.org/abs/2407.21783, 2024.

NVIDIA Corporation. cuBLAS Library User Guide. https://docs.nvidia.com/cuda/cublas/, 2024. CUDA 12.x.

L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 2022.

PyTorch Contributors. Reproducibility — PyTorch documentation. https://pytorch.org/docs/stable/notes/randomness.html, 2024.

M. Research. Phi-4 technical report. https://arxiv.org/abs/2412.08905, 2024.

S. M. Rump, T. Ogita, and S. Oishi. Accurate floating point summation part I: Faithful rounding. *SIAM Journal on Scientific Computing*, 31(1):189–224, 2010.

J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint arXiv:2407.08608*, 2024.

Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Thinking Machines Lab. batch_invariant_ops: Batch-invariant implementations of reduction operations. https://github.com/thinkingmachines/batch-invariant-ops, 2024.

J. Yuan, H. Li, X. Ding, W. Xie, Y.-J. Li, W. Zhao, K. Wan, J. Shi, X. Hu, and Z. Liu. Understanding and mitigating numerical sources of nondeterminism in LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.09501.

L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. SGLang: Efficient execution of structured language model programs. https://arxiv.org/abs/2312.07104, 2024.

A Environment and Reproducibility

A.1 Software Environment

- Python 3.10, PyTorch 2.6.0+cu124, CUDA 12.9
- Transformers 4.48.3, vLLM 0.8.5
- DETERMLLM kernel compiled with `nvcc` (CUDA 12.9, `-O3 -use_fast_math`) targeting `sm_86` (Ampere)

A.2 Kernel Compilation

```
cd FP32
python setup_fixed_algo.py build_ext --inplace
# produces: _gemm_fixed_algo.cpython-310-x86_64-linux-gnu.so
```

A.3 Running the Overhead Benchmark

```
CUDA_VISIBLE_DEVICES=0 \
  python research/run_overhead_bench.py
# Output: research/exp_overhead_bench.json
```

A.4 Running the Log-Probability Experiment

```
CUDA_VISIBLE_DEVICES=0 \
  python research/run_logprob_nondet.py
# Output: research/exp_logprob_nondet.json
```

B cuBLASLt API Details

The `CUBLASLT_MATMUL_PREF_REDUCTION_SCHEME_MASK` attribute accepts a bitmask whose bits correspond to the `cublasLtReductionScheme_t` enum:

Enum value	Bit	Semantics
`CUBLASLT_REDUCTION_SCHEME_NONE`	0x0	Single-CTA per output tile, no split-K
`CUBLASLT_REDUCTION_SCHEME_INPLACE`	0x1	Split-K, atomic in-place reduction
`CUBLASLT_REDUCTION_SCHEME_COMPUTE_TYPE`	0x2	Split-K, reduction in compute type
`CUBLASLT_REDUCTION_SCHEME_OUTPUT_TYPE`	0x4	Split-K, reduction in output type
`CUBLASLT_REDUCTION_SCHEME_MASK`	0x7	Allow all schemes (default)

Setting `mask = 0` is equivalent to `CUBLASLT_REDUCTION_SCHEME_NONE`: only algorithms with no cross-CTA reduction pass the preference filter. This is the key operation in DETERMLLM.

C Proof of Theorem 1 and Corollary 2

Notation

Let $p_k = A_{ik}B_{jk}$ denote the k-th product. We treat each p_k as an FP32 value (the product $A_{ik} \cdot B_{jk}$ is computed in FP32 once the operands are upcast from BF16; the multiplication rounds once but this error is absorbed into the partial products we analyze below). Let ε_{32} be the FP32 unit roundoff and ε_{16} the BF16 unit roundoff, with $\varepsilon_{32} = 2^{-24}$ and $\varepsilon_{16} = 2^{-8}$. Write $\gamma_n = n\varepsilon_{32}/(1 - n\varepsilon_{32})$.

Proof of Theorem 1

Under any permutation π of the indices $\{1, \ldots, K\}$, the FP32 floating-point sum $\hat{C}^{(\pi)} = \widehat{\sum_k} p_{\pi(k)}$ satisfies the standard backward error bound [Higham, 2002, Eq. 4.4]:

$$\left|\hat{C}^{(\pi)} - \sum_{k=1}^{K} p_k\right| \;\le\; \gamma_{K-1} \sum_{k=1}^{K} |p_k| \;\le\; \gamma_K \sum_{k=1}^{K} |p_k|. \tag{3}$$

The bound is order-independent: it holds for every π with the same right-hand side. Applying the triangle inequality to two permutations π_1, π_2,

$$\left|\hat{C}^{(\pi_1)} - \hat{C}^{(\pi_2)}\right| \;\le\; 2\,\gamma_K \sum_{k=1}^{K} |p_k|, \tag{4}$$

which is the claimed bound.

Now let $\hat{y}_\pi = \mathrm{round}_{\mathrm{BF16}}(\hat{C}^{(\pi)})$. BF16 rounding has the property that if two FP32 values x_1, x_2 lie within $\varepsilon_{16}|C_{ij}|/2$ of each other and neither is in the rounding tie-breaking set of C_{ij}, then $\mathrm{round}_{\mathrm{BF16}}(x_1) = \mathrm{round}_{\mathrm{BF16}}(x_2)$. Combining this with (4), a sufficient condition for $\hat{y}_{\pi_1} = \hat{y}_{\pi_2}$ at every π_1, π_2 is

$$2\,\gamma_K \sum_{k=1}^{K} |p_k| \;<\; \frac{\varepsilon_{16}}{2}\,|C_{ij}|, \tag{5}$$

which gives the invariance condition of the theorem. The ties-broken-alike exception is a measure-zero event for continuous inputs and is negligible in practice. □

Proof of Corollary 2

Assume $A_{ik} \sim \mathcal{N}(0,1)$ and $B_{jk} \sim \mathcal{N}(0, 1/K)$, which matches post-LayerNorm activations and Xavier-initialized weights. Then $A_{ik}B_{jk} \sim \mathcal{N}(0, 1/K)$ pairwise independent, so $\mathbb{E}|p_k| = \sqrt{2/(\pi K)}$ and

$$\mathbb{E}\Big[\sum_{k=1}^{K} |p_k|\Big] = K\sqrt{\frac{2}{\pi K}} = \sqrt{\frac{2K}{\pi}}. \tag{6}$$

By the central limit theorem, $C_{ij} \sim \mathcal{N}(0,1)$ for large K, so $\mathbb{E}|C_{ij}| = \sqrt{2/\pi}$. Substituting $\gamma_K \approx K\varepsilon_{32}$ into (5) and taking expectations,

$$2K\varepsilon_{32}\sqrt{\frac{2K}{\pi}} \;<\; \frac{\varepsilon_{16}}{2}\sqrt{\frac{2}{\pi}}, \tag{7}$$

which simplifies to $K^{3/2} < \varepsilon_{16}/(4\varepsilon_{32})$ and yields $K < K_{\max} = (\varepsilon_{16}/(4\varepsilon_{32}))^{2/3}$. Plugging in $\varepsilon_{16}/\varepsilon_{32} = 2^{16}$ gives $K_{\max} = 2^{32/3}/4^{2/3} \approx 7{,}700$.

When the bound is conservative

The proof uses a worst-case Wilkinson bound (3). In practice the achieved error is smaller, for two reasons. First, the rounding errors at each step are signed, not of fixed sign, so cancellation reduces the accumulated error by approximately $\sqrt{K}$ under the stochastic rounding model. Second, the condition number $\kappa(S) = \sum |p_k| / |\sum p_k|$ for post-LayerNorm activations is typically around 30–100 for $K = 4096$, rather than its worst-case upper-bound $\mathcal{O}(\sqrt{K})$. These two factors together make the empirical safe K larger than $K_{\max}$ in many settings (our 8B FFN result at $K = 14{,}336$ is a concrete case: K exceeds the bound but the fix still helps).

Table 8: $K_{\max}$ sweep. "Zero rate" is the fraction of row-0 output cells that match bit-for-bit between $M=2$ and $M=32$. DETERMLLM stays at or near 100% across the whole range, including K well beyond $K_{\max} \approx 7{,}700$.

K	BF16 zero rate	DETERMLLM zero rate	BF16 ℓ_∞	DETERMLLM ℓ_∞
2,048	99.9%	99.9%	4.0×10^{-4}	4.0×10^{-4}
4,096	99.9%	99.9%	4.9×10^{-3}	4.9×10^{-3}
5,120	99.8%	99.8%	7.5×10^{-3}	7.5×10^{-3}
6,144	100 %	100 %	0.0	0.0
7,168	58.3%	**99.8%**	1.6×10^{-2}	7.4×10^{-3}
7,680	73.0%	**99.7%**	1.6×10^{-2}	1.0×10^{-2}
8,192	58.3%	**99.7%**	1.6×10^{-2}	9.4×10^{-3}
10,240	99.6%	99.6%	9.4×10^{-3}	9.4×10^{-3}
12,288	61.0%	**100 %**	1.6×10^{-2}	0.0
14,336	59.0%	**99.5%**	1.7×10^{-2}	1.4×10^{-2}
16,384	58.9%	**99.4%**	1.6×10^{-2}	1.2×10^{-2}

D $K_{\max}$ Sweep: Full Data

Table 8 gives the full K-sweep referenced in Section 4.4. Each row averages 10 trials of a synthetic $M\times K\times K$ GEMM. We run $M=2$ and $M=32$, both inside the GEMM code path, and report the fraction of row-0 output cells that match bit-for-bit between the two batch sizes.

BF16 drops sharply at $K\in\{7{,}168,\ 7{,}680,\ 8{,}192\}$ and stays below 73% for most $K\geq 7{,}168$. The two intermediate rows ($K=6{,}144$ and $K=10{,}240$) show that cuBLAS happens to select the same non-split-K algorithm for both $M=2$ and $M=32$ at those dimensions, which is consistent with the split-K heuristic being (M,N,K)-specific rather than monotonic in K.

E Row-Major Layout Bug: Formal Analysis

For a matrix $A\in\mathbb{R}^{M\times K}$ stored row-major, element (i,j) resides at byte offset $(i\cdot K+j)\cdot$ sizeof(elem). The cuBLASLt API treats the *leading dimension* as the stride between consecutive rows (row-major) or columns (column-major). Without setting `CUBLASLT_ORDER_ROW`, cuBLASLt defaults to column-major with $ld=K$ (the value passed at layout creation). Under column-major, element (i,j) is addressed at offset $i+j\cdot K$. For $M=2, K=4096$:

$$\text{max address (col-major)} = (M-1)+(K-1)\cdot K = 1+4095\cdot 4096 \approx 16.7\text{M elements},$$

far exceeding the $M\cdot K=8192$ element buffer. The kernel writes out-of-bounds silently (GPU memory access is asynchronous and does not immediately trap), corrupting adjacent tensors. The next cuBLAS call then fails with `CUBLAS_STATUS_INTERNAL_ERROR` because the workspace or weight tensors have been overwritten.

F Batched-Prefill KV-Cache Experiment (Confounding Setting)

Table 9 shows the confounding batched-prefill setting where all N requests are prefilled together. Prefill attention ($M=L$) is unpatched, contaminating the KV cache before decode begins. DETERMLLM provides only marginal improvement here, confirming this is not a meaningful production evaluation.

G 50-Prompt Broader Evaluation

Table 10 replicates the 1B decomposition on 50 diverse prompts. Of the 33 prompts affected by BF16 non-determinism, DETERMLLM fully resolves 24 (73%). The 10 remaining include 9 pre-existing and 1 counter-direction case (prompt 9: BF16-stable but FP32-unstable), attributable to the

Table 9: Batched prefill + KV decode (Llama-3.2-1B, not production-representative).

Condition	All match?	Tot. tok. MM	Avg $\lvert\Delta \log P\rvert$
A: BF16 baseline	×	521	0.0661 nats
B: DETERMLLM (F.linear)	×	486	0.0582 nats
C: DETERMLLM (+ attn)	×	484	0.0626 nats

FFN down-projection ($K = 8{,}192 > K_{\max}$). The 24 resolved prompts accounted for 1,262 token mismatches in A (42% of baseline total).

Table 10: 50-prompt evaluation (Llama-3.2-1B, full re-computation, batch sizes 2–64). Of 33 affected prompts, DETERMLLM fully resolves 24 (73%). Padding fix (D) identical to B, as expected in the full re-computation setting.

Condition	Prompts with any MM	Tot. tok. MM	Avg $\lvert\Delta \log P\rvert$
A: BF16 baseline	33 / 50	2,981	0.0849 nats
B: DETERMLLM (F.linear)	**10 / 50**	**1,800**	**0.0571** nats

H Kernel Implementations

H.1 cuBLASLt Backend: Two-Phase Heuristic

Listing 2: `gemm_fixed_algo.cu`.

```
// FP32 accumulation for all reductions
cublasLtMatmulDescCreate(&desc, CUBLAS_COMPUTE_32F, CUDA_R_32F);

// Phase 1: no split-K (optimal for small M, single CTA per output tile)
mask = CUBLASLT_REDUCTION_SCHEME_NONE;  // = 0x0
cublasLtMatmulAlgoGetHeuristic(..., mask, 1, &algo, &nFound);

// Phase 2: FP32 inter-CTA reduction (allows split-K for large M;
// batch-invariant by Theorem 1: FP32 error << BF16 quantization step)
if (!nFound) {
    mask = CUBLASLT_REDUCTION_SCHEME_COMPUTE_TYPE;  // = 0x2
    cublasLtMatmulAlgoGetHeuristic(..., mask, 1, &algo, &nFound);
}

// Consistent FP32 fallback (same precision argument applies)
if (!nFound)
    return torch::mm(A.to(kFloat), B.t().to(kFloat)).to(kBFloat16);
```

H.2 Triton Backend: Fixed-Plan GEMM

Listing 3: Core of `triton_det_gemm.py`.

```
@triton.autotune(configs=_autotune_configs(),
                 key=['M_class', 'N', 'K'])   # cached per (M_class, N, K
                     )
@triton.jit
def _det_gemm_kernel(a_ptr, b_ptr, c_ptr, M, N, K, M_class,
                     stride_am, stride_ak, stride_bn, stride_bk,
                     stride_cm, stride_cn,
                     BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr,
                     BLOCK_K: tl.constexpr, GROUP_M: tl.constexpr):
    # One program per output tile. No split-K: the K-reduction below
```

```
    # is a single sequential loop, so the per-element reduction path
    # is byte-identical regardless of M.
    pid_m, pid_n = _group_major_schedule(tl.program_id(0), M, N,
                                         BLOCK_M, BLOCK_N, GROUP_M)
    offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    offs_k = tl.arange(0, BLOCK_K)
    a_ptrs = a_ptr + offs_m[:, None] * stride_am + offs_k[None, :] *
        stride_ak
    b_ptrs = b_ptr + offs_n[:, None] * stride_bn + offs_k[None, :] *
        stride_bk

    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
    for k in range(0, tl.cdiv(K, BLOCK_K)):
        a = tl.load(a_ptrs, mask=..., other=0.0)    # BF16 [M, K]
        b = tl.load(b_ptrs, mask=..., other=0.0)    # BF16 [N, K]
        acc += tl.dot(a, tl.trans(b))               # HMMA.16816.F32
        a_ptrs += BLOCK_K * stride_ak
        b_ptrs += BLOCK_K * stride_bk

    tl.store(c_ptr + offs_m[:, None] * stride_cm + offs_n[None, :] *
        stride_cn,
             acc.to(tl.bfloat16), mask=...)
```

The correctness property is straightforward: `SPLIT_K` $= 1$ plus a fixed `(BLOCK_M, BLOCK_N, BLOCK_K)` chosen by autotune per (M_{class}, N, K) means each output element's reduction instructions are the same across batch sizes that fall in the same bucket. Triton's `tl.dot` on Ampere emits `HMMA.16816.F32` with BF16 operands and FP32 accumulator, so precision is FP32 inside and BF16 only on the final `tl.store`. Ampere, Hopper, and Blackwell all support this instruction.

I Model-Level Logit Perturbation

Table 11: Model-level logit perturbation (Llama-3.1-8B, FlashAttention backend, $bs = 1$ reference vs. batch size N). With DETERMLLM, linear-layer contributions collapse to zero; residual attention non-determinism is negligible at $bs = 8$.

Batch size	Max $\lvert\Delta\text{logit}\rvert$	Mean $\lvert\Delta\text{logit}\rvert$	Logits affected (%)
$bs = 2$	0.219	0.022	78.5
$bs = 8$	**26.81**	2.57	99.8
$bs = 8$ (DETERMLLM)	**0.0**	0.0	0.0

J Why FP32 Upcast Does Not Yield Batch Invariance

A reasonable hypothesis is that LayerCast (which runs all F.linear in FP32) should automatically be bit-invariant across batch sizes, since FP32 has far tighter rounding than BF16. Two diagnostic scripts rule this out empirically.

F.linear shapes. `research/diagnose_triton_bs_invariance.py` builds a random BF16 input $x \in \mathbb{R}^{1\times K}$ and weight $W \in \mathbb{R}^{N\times K}$ for each of DeepSeek-7B's six projection shapes, computes $y_1 = xW^\top$ and $y_8 = (x.\text{repeat}(8,1))W^\top$, and asks whether $y_1[0] \equiv y_8[0]$ bit-for-bit. Under stock FP32 cuBLAS, $5/6$ shapes fail: $(3584, 3584)$ Q/O-proj drifts by 1.5×10^{-5}, $(3584, 18944)$ gate/up by 2.0×10^{-3}, $(18944, 3584)$ down by 4.9×10^{-4}, and $(3584, 151\,936)$ lm_head by 3.9×10^{-3}. Only the GQA K/V projection $(3584, 512)$ passes—because its small N

routes to a different, non-split-K heuristic. The Triton backend passes all six by construction. Setting `torch.use_deterministic_algorithms(True)` and `CUBLAS_WORKSPACE_CONFIG=:4096:8` does not help: FP32 still drifts by 4.6×10^{-5} on the Q-proj shape.

Attention shapes. `research/diagnose_fp32_matmul_bs_invariance.py` sweeps decode-phase $\mathrm{Q} \otimes \mathrm{K}^{\top}$ at $S = 1, \ldots, 300$ context tokens with $H = 28$, $D = 128$. 54 of 300 context lengths produce bs-dependent output under FP32 upcast; the drift is mostly at the FP32 rounding floor (10^{-7}–10^{-5}) but occasionally spikes to 3×10^{-2} when cuBLAS flips to a split-K algorithm. Short generations (our initial 64-token smoke test) coincidentally avoid the spiking S values; longer generations (1024 tokens) reliably hit them.

Consequence. LayerCast as described by Yuan et al. and as faithfully implemented here (F.linear-only FP32 upcast) does not produce bit-identical token sequences across $bs \in \{1, 8, 32\}$ in our stress regime. Patching `torch.matmul` with an FP32 upcast on top (LayerCast + attention upcast) only marginally improves Div_Index, because the cuBLAS split-K issue affects FP32 matmul too. The only path we found to bit-exact cross-bs output is a fixed-plan kernel (our Triton backend). This is consistent with Yuan et al.'s FP32 numbers being measured under a narrower bs range (theirs: $\{8, 16\}$; ours: $\{1, 8, 32\}$) and shorter generations.